

Responder-assisted LightGBM/CatBoost Strategy

Training data modes

The trainer supports two data-loading backends:

- `--training-data-mode out-of-core` (default): read cached shards through `lightgbm.Sequence`.
- `--training-data-mode in-memory`: concatenate the current training and validation ranges into contiguous `float32` matrices before fitting.

Use the in-memory backend when the machine has enough RAM:

```
python examples/responder_assisted_lgb_catboost_strategy/train.py --data-root
data --work-dir examples/responder_assisted_lgb_catboost_strategy/work --model-
dir examples/responder_assisted_lgb_catboost_strategy/model --training-data-
mode in-memory
```

Both modes use identical time splits and features. OOF responder folds remain time-separated to prevent leakage. The in-memory mode reports each matrix's row count, feature count, and size; reserve additional RAM for labels, weights, LightGBM bins, and training workspace.

Target experiment suites

The default `--experiment-suite next-step` trains these target ablations:

- **A**: raw features only.
- **C4**: raw features plus all four responder predictions.
- **C2**: raw features plus `responder_02/03`.
- **T60**: C2 plus `rolling_std60` and `minus_ema60`.
- **T20_60**: C2 plus 20/60-window volatility and EMA-deviation groups.
- **TZ**: T20_60 plus 20/60-window historical z-scores.

Each experiment reports the overall validation score and four contiguous validation-time scores. The selected model metadata records the exact cached column indices and responder subset used by inference.

Run only selected experiments:

```
python examples/responder_assisted_lgb_catboost_strategy/train.py --data-root
data --work-dir examples/responder_assisted_lgb_catboost_strategy/work --model-
dir examples/responder_assisted_lgb_catboost_strategy/model --target-
experiments C2,T60,T20_60
```

Run the original A/B/C/D comparison:

```
python examples/responder_assisted_lgb_catboost_strategy/train.py --data-root
data --work-dir examples/responder_assisted_lgb_catboost_strategy/work --model-
dir examples/responder_assisted_lgb_catboost_strategy/model --experiment-suite
legacy
```

The default temporal fallback no longer creates `delta1` or `xs_rank_delta1`. Cache schema version 5 forces older caches and OOF models to be rebuilt so the removed columns cannot silently survive.

这个目录用于逐步实现利用 `responder_*` 辅助训练的 LightGBM/CatBoost 策略。

当前第一阶段只分析 responder 与最终 `target` 的关系，不会训练模型，也不会把真实 responder 当作推理特征。正式测试不提供 `responder_*`，后续阶段将使用严格时间 OOF 的 responder 预测进行 stacking。

第一阶段：Responder 相关性分析

```
python examples/responder_assisted_lgb_catboost_strategy/analyze_responders.py \
--data-root data \
--output-dir examples/responder_assisted_lgb_catboost_strategy/analysis
```

Windows PowerShell 单行命令：

```
python examples/responder_assisted_lgb_catboost_strategy/analyze_responders.py
--data-root data --output-dir
examples/responder_assisted_lgb_catboost_strategy/analysis
```

默认最多抽样 2,000,000 行。小规模冒烟测试：

```
python examples/responder_assisted_lgb_catboost_strategy/analyze_responders.py
--data-root data --output-dir
examples/responder_assisted_lgb_catboost_strategy/analysis_smoke --max-rows
100000 --time-bins 5
```

输出文件：

- `responder_summary.csv`：综合相关性和稳定性排名；
- `responder_period_correlations.csv`：连续时间段内的加权相关性；
- `responder_time_ic.csv`：每个 `time_id` 的截面 Spearman IC；
- `analysis_report.json`：运行信息和前十名 responder。

`responder_summary.csv` 包含：

- 全局加权 Pearson、普通 Pearson 和 Spearman；
- 按 `time_id` 去均值后的相关性；
- 按 `asset_id` 去均值后的相关性；
- 逐时点截面 Spearman IC 的均值、标准差、ICIR 和方向稳定性；
- 连续时间段相关性的均值、波动、方向稳定性和最近一期相关性；
- 用于初筛的 `screening_score`。

`screening_score` 只用于缩小候选 responder 范围。最终是否采用某个 responder，仍需在后续阶段通过严格时间 OOF responder 预测和 target 样本外增益判断。

第二阶段：Responder OOF Stacking + LightGBM

当前实现使用以下四个 responder：

```
responder_03, responder_28, responder_29, responder_02
```

模型输入还会从重要性最高的 30 个原始 feature 生成 330 个时序特征，每个原始 feature 对应：`lag1`、`lag5`、`delta1`、`delta5`、历史 EMA5、历史 EMA20、相对 EMA20 偏离、历史 20 期标准差、历史 z-score、当前截面 rank 和截面 rank 相对上一期的变化。所有历史统计都只使用当前 `time_id` 之前的数据。

根据首次消融的重要性结果，当前路由会移除 `delta1` 和 `xs_rank_delta1`，并在已有中期类型上自动增加 `lag20`、`delta20`、EMA60、相对 EMA60 偏离、60期滚动标准差和60期历史 z-score。`lag1` 与 EMA5 仍有一定 gain，因此暂时保留。

推荐先运行无监督时序类型分析，让每个 feature 只生成适合自己的派生特征：

```
python
examples/responder_assisted_lgb_catboost_strategy/analyze_feature_temporal_type
s.py --data-root data --output-dir
examples/responder_assisted_lgb_catboost_strategy/analysis
```

该脚本输出 `feature_temporal_statistics.csv`、`feature_temporal_routes.csv` 和 `temporal_feature_plan.json`。训练脚本默认读取 `analysis/temporal_feature_plan.json`；也可以使用 `--temporal-plan` 指定其他计划。计划不存在时才回退到前 30 个 feature 统一生成 11 种特征。

训练链路：

1. 按完整 `time_id` 流式读取 Parquet，避免截断资产截面；
2. 将原始 `feature_* + asset_id` 写成磁盘 NumPy shards；
3. 使用 expanding-window 时间折训练 responder LightGBM，生成无泄漏 OOF 预测；
4. 用 OOF `responder_hat` 和原始特征联合训练 target LightGBM；
5. 在验证截止点之前的全部数据上重训四个 responder 模型；

6. 推理时先预测 responder, 再预测 target。

LightGBM 通过磁盘分片 `Sequence` 联合构造 Dataset。它不会一次性创建完整的 float32 特征矩阵；模型看到的是所选时间区间内的全部分片，而不是逐分片重复 `fit`。LightGBM 内部仍会为量化后的 Dataset、梯度和树结构分配内存。

```
python examples/responder_assisted_lgb_catboost_strategy/train.py --data-root
data --work-dir examples/responder_assisted_lgb_catboost_strategy/work --model-
dir examples/responder_assisted_lgb_catboost_strategy/model
```

首次运行会建立磁盘缓存，后续运行默认复用。数据或特征定义变化后添加 `--rebuild-cache`。缓存可能占用数十 GB 磁盘空间。

断点恢复并跳过已有模型：

```
python examples/responder_assisted_lgb_catboost_strategy/train.py --data-root
data --work-dir examples/responder_assisted_lgb_catboost_strategy/work --model-
dir examples/responder_assisted_lgb_catboost_strategy/model --skip-existing-
models
```

启用该选项后，程序会复用 `work/oof_models` 中已有的 OOF 折模型以及 `model` 中的最终模型，仅训练缺失文件。如果五个最终模型和 `metadata.json` 全部存在，程序会在启动时直接输出已有 metadata 并结束。训练过程中会输出缓存分片、OOF 折、各 responder、分批预测和 target 模型的进度。

默认 `--ablation-mode all` 会训练并比较四个 target 模型：

- A: 原始 feature + asset_id;
- B: A + 路由时序特征;
- C: A + responder_hat;
- D: A + 路由时序特征 + responder_hat。

验证分数最高的变体会保存为正式 `target_lightgbm.txt`。详细结果写入 `model/ablation_report.json`，逐模型 gain/split importance 写入 `model/target_feature_importance.csv`。也可通过 `--ablation-mode A` 等只训练指定变体。

审计高 R^2 responder:

```
python examples/responder_assisted_lgb_catboost_strategy/audit_responders.py --
work-dir examples/responder_assisted_lgb_catboost_strategy/work --model-dir
examples/responder_assisted_lgb_catboost_strategy/model --output-dir
examples/responder_assisted_lgb_catboost_strategy/audit
```

该脚本检查缓存维度和时间顺序，计算每个输入与 `responder_02/03` 的训练/验证加权相关性、单特征线性样本外 R^2 、asset 固定均值基线，并在 LightGBM 可用时执行标签

打乱负对照。详细特征结果和 JSON 报告写入 `audit/`。

本地顺序推理验证：

```
python3 timeseries_api/run_timeseries_api.py --data-root data --strategy-dir
examples/responder_assisted_lgb_catboost_strategy --output
examples/responder_assisted_lgb_catboost_strategy/submission.csv
```